

PROLET

Sepideh Pashayan

402103242

Dr.Firoozeh

هدف پروژه :

هدف این پروژه، ساخت یک سیستم دسته بندی تصویر دودویی با استفاده از SVM است که بتواند به طور خودکار تصاویر خودرو و موتورسیکلت را از یکدیگر تشخیص دهد.

برای این منظور، هر تصویر به یک بردار ویژگی عددی تبدیل شده و به مدل SVM داده می شود. چندین تابع کرنل مختلف linear و RBF آموزش داده شده و با یکدیگر مقایسه می شوند تا بهترین پیکربندی پیدا شود. در نهایت مدل روی داده های تست ارزیابی شده و رفتار آن بررسی می گردد.

دیتاست : Car-Bike Classification

دیتاست مورد استفاده در این پروژه، Car-Bike Dataset است که از سایت Kaggle تهیه شده است.

این دیتاست شامل ۴۰۰۰ تصویر در دو کلاس متوازن است:

Bike: 2000 pictures

Car: 2000 pictures

دیتاست کاملاً متوازن است، یعنی هر دو کلاس تعداد یکسانی از نمونه ها دارند. این موضوع از تمایل مدل به یک کلاس خاص جلوگیری می کند.

```
import os      Pin selection to current chat prompt (Ctrl+Alt+X) | Don't sho
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import time
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

✓ 1m 51.3s

در این پروژه از کتابخانه‌های زیر استفاده شده است:

os:

برای کار با مسیر فایل‌ها و فولدرها

numpy:

برای پردازش آرایه‌های عددی و تبدیل تصاویر به بردار

PIL (Pillow):

برای باز کردن، تغییر اندازه و پردازش تصاویر

matplotlib:

برای رسم نمودار و نمایش تصاویر

scikit-learn:

شامل موارد زیر:

StandardScaler:

برای مقیاس‌بندی ویژگی‌ها

train_test_split:

برای تقسیم داده به train و test

SVC:

مدل اصلی SVM

accuracy_score, confusion_matrix, classification_report:

برای ارزیابی مدل

seaborn:

به شکل بصری Confusion Matrix برای رسم

time:

برای اندازه‌گیری زمان آموزش هر مدل

warnings:

برای حذف پیام‌های هشدار غیرضروری

```
dataset_path = os.path.join("archive", "Car-Bike-Dataset")
✓ 0.0s

classes = ['Bike', 'Car']
✓ 0.7s

for cls in classes:
    folder = os.path.join(dataset_path, cls)
    files = os.listdir(folder)
    print(f"{cls}: {len(files)} pictures")
✓ 0.4s

Bike: 2000 pictures
Car: 2000 pictures
```

تعریف مسیر و کلاس‌ها

مسیر دیتاست به صورت نسبی تعریف شده است تا کد روی هر سیستمی قابل اجرا باشد. دو کلاس Bike و Car به عنوان کلاس‌های هدف تعیین شدند.

بررسی تعداد تصاویر

پیش از شروع پردازش، تعداد تصاویر هر کلاس بررسی شد:

کلاس	تعداد تصویر
Bike	۲۰۰۰
Car	۲۰۰۰

همان‌طور که مشاهده می‌شود، دیتاست کاملاً متوازن است و نیازی به تکنیک‌های oversampling یا undersampling نیست.

هر تصویر از دیتاست طی مراحل زیر پردازش شد:

۱. تبدیل به Grayscale
تصاویر رنگی از ۳ کانال RGB به ۱ کانال grayscale تبدیل شدند. این کار تعداد ویژگی‌ها را به یک سوم کاهش می‌دهد و سرعت آموزش را افزایش می‌دهد.

۲. تغییر اندازه Resize
همه تصاویر به اندازه یکسان 64×64 پیکسل تبدیل شدند تا ورودی مدل همیشه یک اندازه باشد.

۳. تبدیل به بردار Flatten
ماتریس 64×64 هر تصویر به یک بردار ۴۰۹۶ تایی تبدیل شد، زیرا SVM با بردارهای یک بعدی کار می‌کند.

```
IMG_SIZE = 64
X = []
y = []
for label, cls in enumerate(classes):
    folder = os.path.join(dataset_path, cls)
    files = os.listdir(folder)

    for file in files:
        img_path = os.path.join(folder, file)
        try:
            img = Image.open(img_path).convert('L')
            img = img.resize((IMG_SIZE, IMG_SIZE))
            img_array = np.array(img).flatten()
            X.append(img_array)
            y.append(label)
        except:
            pass
X = np.array(X)
y = np.array(y)

print(f"X.shape: {X.shape}")
print(f"y.shape: {y.shape}")
```

✓ 4m 8.4s

X.shape: (4000, 4096)
y.shape: (4000,)

متغیر	شکل	توضیح
X	(4000, 4096)	۴۰۰۰ تصویر، هر کدام ۴۰۹۶ ویژگی
y	(4000,)	۴۰۰۰ برچسب 1=Car, 0=Bike

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"train: {X_train.shape}")
print(f"test: {X_test.shape}")
```

✓ 1.7s

train: (3200, 4096)
test: (800, 4096)

تقسیم داده به Train و Test
داده‌ها با نسبت ۸۰/۲۰ تقسیم شدند:

مجموعه	تعداد	درصد
Train	۳۲۰۰	۸۰٪
Test	۸۰۰	۲۰٪

پارامتر `stratify=y` تضمین می‌کند که نسبت Bike به Car در هر دو مجموعه یکسان باشد.

قیاس‌بندی با **StandardScaler**

SVM به مقیاس ویژگی‌ها بسیار حساس است. دلیل این حساسیت این است که SVM بر اساس فاصله بین نقاط کار می‌کند. اگر یک ویژگی مقادیر بسیار بزرگ تری نسبت به بقیه داشته باشد، آن ویژگی تأثیر بیشتری روی مدل می‌گذارد و نتیجه را منحرف می‌کند.

StandardScaler هر ویژگی را به گونه‌ای تبدیل می‌کند که میانگین=۰ و انحراف معیار=۱ داشته باشد.

نکته مهم `fit_transform`:

فقط روی داده train اعمال شد و روی test فقط `transform` استفاده شد. دلیل این است که نباید اطلاعات داده test در مرحله آموزش تأثیر بگذارد.


```

kernels = [
    {'kernel': 'linear', 'C': 0.1},
    {'kernel': 'linear', 'C': 1},
    {'kernel': 'linear', 'C': 10},
    {'kernel': 'rbf', 'C': 1, 'gamma': 'scale'},
    {'kernel': 'rbf', 'C': 1, 'gamma': 0.001},
    {'kernel': 'rbf', 'C': 10, 'gamma': 'scale'},
    {'kernel': 'rbf', 'C': 10, 'gamma': 0.001},
]

results = []
models = {}

for params in kernels:
    print(f"Training: {params} ...")
    start = time.time()
    model = SVC(**params)
    model.fit(X_train, y_train)
    elapsed = time.time() - start

    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))

    key = str(params)
    models[key] = model
    results.append(**params, 'train_acc': train_acc, 'test_acc': test_acc)
    print(f"train: {train_acc:.3f} | test: {test_acc:.3f} | Time: {elapsed:.1f}s")
    print("-" * 50)

best = max(results, key=lambda x: x['test_acc'])
print(f"\nBest model: kernel={best['kernel']}, C={best['C']}, test_acc={best['test_acc']:.3f}")

```

✓ 49m 5.3s

```

Training: {'kernel': 'linear', 'C': 0.1} ...
train: 1.000 | test: 0.724 | Time: 113.1s
-----
Training: {'kernel': 'linear', 'C': 1} ...
train: 1.000 | test: 0.723 | Time: 181.1s
-----
Training: {'kernel': 'linear', 'C': 10} ...
train: 1.000 | test: 0.723 | Time: 201.9s
-----
Training: {'kernel': 'rbf', 'C': 1, 'gamma': 'scale'} ...
train: 0.933 | test: 0.830 | Time: 217.7s
-----
Training: {'kernel': 'rbf', 'C': 1, 'gamma': 0.001} ...
train: 0.997 | test: 0.829 | Time: 184.6s
-----
Training: {'kernel': 'rbf', 'C': 10, 'gamma': 'scale'} ...
train: 1.000 | test: 0.835 | Time: 192.8s
-----
Training: {'kernel': 'rbf', 'C': 10, 'gamma': 0.001} ...
train: 1.000 | test: 0.836 | Time: 203.1s
-----

Best model: kernel=rbf, C=10, test_acc=0.836

```

پارامترهای آزمایش شده

برای یافتن بهترین مدل، دو کرنل linear و RBF با مقادیر مختلف C و gamma آزمایش شدند:

بهترین مدل: RBF با C=10 و gamma=0.001 با دقت تست ۸۳/۶٪

رئل	C	gamma	Train	Test	زمان
linear	0.1	-	1.000	0.724	113s
linear	1	-	1.000	0.723	181s
linear	10	-	1.000	0.723	202s
rbf	1	scale	0.933	0.830	218s
rbf	1	0.001	0.997	0.829	185s
rbf	10	scale	1.000	0.835	193s
rbf	10	0.001	1.000	0.836	203s

نقش پارامتر C

پارامتر C کنترل می‌کند که مدل چقدر به اشتباهات آموزشی حساس باشد:

• C کوچک: مدل ساده‌تر، ممکن است underfitting داشته باشد

• C بزرگ: مدل سخت‌گیرتر، ممکن است overfitting داشته باشد

در نتایج مشاهده می‌شود که برای کرنل linear، تغییر C تأثیر چندانی روی دقت تست نداشت.

نقش پارامتر gamma

پارامتر gamma فقط در کرنل RBF وجود دارد و کنترل می‌کند که هر نمونه چقدر روی مرز تصمیم تأثیر بگذارد:

Gamma بزرگ: هر نمونه فقط روی همسایه‌های نزدیک تأثیر می‌گذارد ← overfitting

Gamma کوچک: هر نمونه روی ناحیه وسیع‌تری تأثیر می‌گذارد ← underfitting

چرا RBF بهتر از linear عمل کرد؟

کرنل linear یک مرز تصمیم خطی رسم می‌کند. اما داده‌های تصویری معمولاً از

نظر ویژگی‌ها غیرخطی هستند، یعنی نمی‌توان آن‌ها را با یک خط از هم جدا کرد.

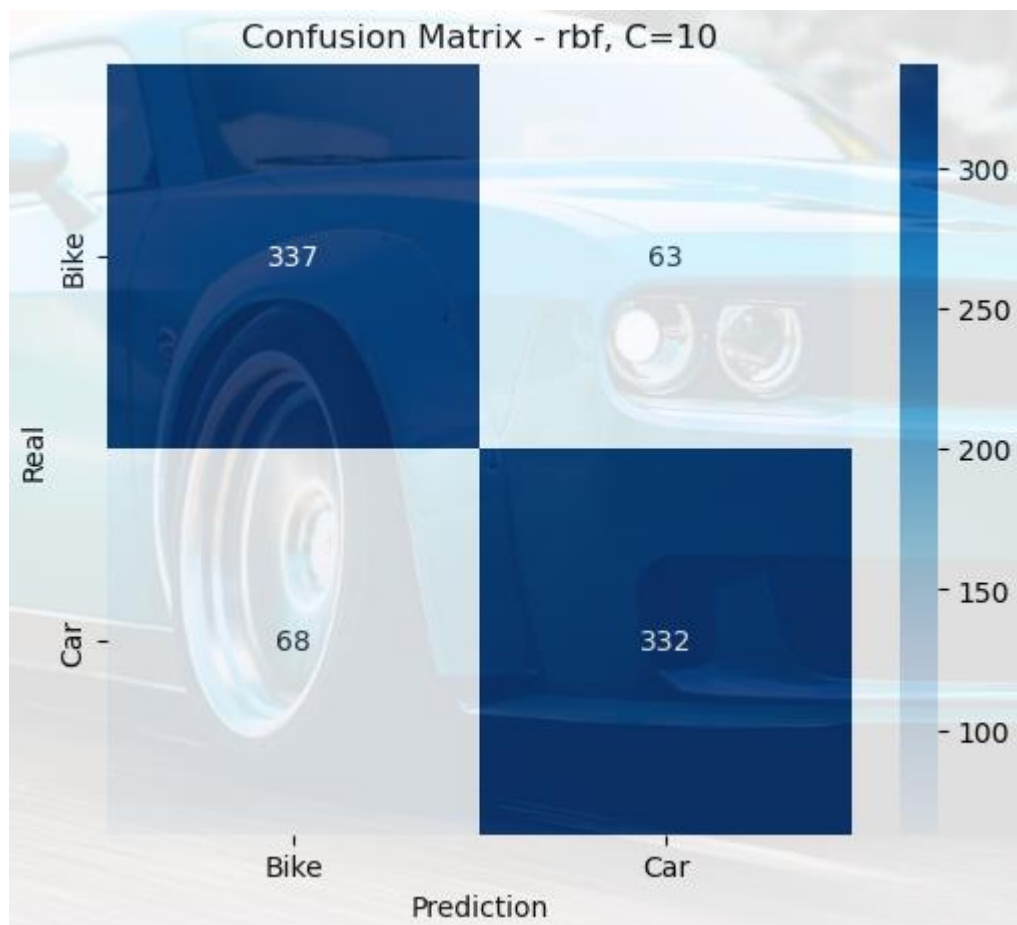
کرنل RBF با تبدیل داده‌ها به فضای بالاتر، مرز تصمیم غیرخطی رسم می‌کند و بهتر عمل می‌کند.


```
Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again
best = max(results, key=lambda x: x['test_acc'])
best_model = models[str({k: v for k, v in best.items() if k not in ['train_acc', 'train_loss']})]
y_pred = best_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Bike', 'Car'],
            yticklabels=['Bike', 'Car'])
plt.xlabel('Prediction')
plt.ylabel('Real')
plt.title(f"Confusion Matrix - {best['kernel']}, C={best['C']}")
plt.show()

print(classification_report(y_test, y_pred, target_names=['Bike', 'Car']))
```

✓ 1m 7.3s



	precision	recall	f1-score	support
Bike	0.83	0.84	0.84	400
Car	0.84	0.83	0.84	400
accuracy			0.84	800
macro avg	0.84	0.84	0.84	800
weighted avg	0.84	0.84	0.84	800

ارزیابی بهترین مدل

بهترین مدل: RBF با $C=10$ و $\gamma=0.001$

Confusion Matrix

	Bike پیش‌بینی	Car پیش‌بینی
Bike واقعی	337 ✓	63 ✗
Car واقعی	68 ✗	332 ✓

- 337 تصویر Bike درست تشخیص داده شد
- 332 تصویر Car درست تشخیص داده شد
- 63 تصویر Bike اشتباهاً Car تشخیص داده شد
- 68 تصویر Car اشتباهاً Bike تشخیص داده شد

معیارهای ارزیابی

دقت کلی: 84% (Accuracy)

کلاس	Precision	Recall	F1-Score
Bike	0.83	0.84	0.84
Car	0.84	0.83	0.84
میانگین	0.84	0.84	0.84

مدل دقت ۸۴٪ روی داده‌های تست به دست آورد که برای یک مدل مبتنی بر پیکسل خام بدون استخراج ویژگی پیشرفته، نتیجه قابل قبولی است. تعادل خوبی بین Precision و Recall در هر دو کلاس مشاهده می‌شود که نشان می‌دهد مدل به هیچ کلاسی تمایل ندارد.

```
print("Analysis of Overfitting:\n")
for r in results:
    diff = r['train_acc'] - r['test_acc']
    kernel = r['kernel']
    C = r['C']
    gamma = r.get('gamma', '-')
    print(f"kernel={kernel}, C={C}, gamma={gamma}")
    print(f"  train={r['train_acc']:.3f} | test={r['test_acc']:.3f} | Difference={diff:.3f}")
    if diff > 0.05:
        print(f"Overfit")
    else:
        print(f"Balanced")
    print()
```

✓ 0.0s

Analysis of Overfitting:

kernel=linear, C=0.1, gamma=-
 train=1.000 | test=0.724 | Difference=0.276
Overfit

kernel=linear, C=1, gamma=-
 train=1.000 | test=0.723 | Difference=0.277
Overfit

kernel=linear, C=10, gamma=-
 train=1.000 | test=0.723 | Difference=0.277
Overfit

kernel=rbf, C=1, gamma=scale
 train=0.933 | test=0.830 | Difference=0.103
Overfit

kernel=rbf, C=1, gamma=0.001
 train=0.997 | test=0.829 | Difference=0.168
Overfit

kernel=rbf, C=10, gamma=scale
 train=1.000 | test=0.835 | Difference=0.165
Overfit

...
kernel=rbf, C=10, gamma=0.001
 train=1.000 | test=0.836 | Difference=0.164
Overfit

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

تحلیل بیش‌برازش (Overfitting) نتایج مقایسه Train و Test

وضعیت	اختلاف	Test	Train	gamma	C	کرنل
Overfit	0.276	0.724	1.000	-	0.1	linear
Overfit	0.277	0.723	1.000	-	1	linear
Overfit	0.277	0.723	1.000	-	10	linear
Overfit	0.103	0.830	0.933	scale	1	rbf
Overfit	0.168	0.829	0.997	0.001	1	rbf
Overfit	0.165	0.835	1.000	scale	10	rbf
Overfit	0.164	0.836	1.000	0.001	10	rbf

در تمامی مدل‌ها overfitting مشاهده شد، اما شدت آن متفاوت است:
 کرنل linear بدترین وضعیت را دارد دقت train برابر ۱۰۰٪ ولی دقت test تنها ۷۲٪ است که اختلافی حدود ۲۷٪ نشان می‌دهد. این نشان می‌دهد مدل linear داده‌های train را حفظ کرده ولی قدرت تعمیم ضعیفی دارد.
 در مقابل، کرنل RBF با $C=1$ و $\text{gamma}=\text{scale}$ کمترین اختلاف (۰/۱۰۳) را دارد و بهترین تعادل بین train و test را نشان می‌دهد.

دلیل اصلی overfitting استفاده از پیکسل خام به عنوان ویژگی است. هر تصویر ۴۰۹۶ ویژگی دارد که بسیاری از آن‌ها نویز هستند و اطلاعات مفیدی ندارند. استفاده از روش‌های استخراج ویژگی پیشرفته‌تر مانند HOG می‌تواند این مشکل را کاهش دهد.


```

from sklearn.neighbors import KNeighborsClassifier

knn_results = []

for k in [3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, knn.predict(X_train))
    test_acc = accuracy_score(y_test, knn.predict(X_test))

    knn_results.append({'k': k, 'train_acc': train_acc, 'test_acc': test_acc})
    print(f"KNN (k={k}): train={train_acc:.3f} | test={test_acc:.3f}")

best_knn = max(knn_results, key=lambda x: x['test_acc'])
print(f"\nBest KNN: k={best_knn['k']}, test_acc={best_knn['test_acc']:.3f}")
print(f"Best SVM: kernel={best['kernel']}, C={best['C']}, test_acc={best['test_acc']:.3f}")

if best_knn['test_acc'] > best['test_acc']:
    print("\nKNN performed better than SVM!")
else:
    print("\nSVM performed better than KNN!")

```

KNN (k=3): train=0.872 | test=0.715
 KNN (k=5): train=0.837 | test=0.713
 KNN (k=7): train=0.827 | test=0.708

Best KNN: k=3, test_acc=0.715
 Best SVM: kernel=rbf, C=10, test_acc=0.836

SVM performed better than KNN!

مقایسه با KNN بخش امتیازی

نتایج KNN

بهترین KNN: k=3 با دقت تست 71.5%

k	Train	Test
3	0.872	0.715
5	0.837	0.713
7	0.827	0.708

مقایسه بهترین KNN با بهترین SVM

SVM با اختلاف ۱۲/۱٪ بهتر از KNN عمل کرد.

مدل	پارامتر	دقت تست
KNN	k=3	0.715
SVM	rbf, C=10, gamma=0.001	0.836

KNN برای داده‌های تصویری با ابعاد بالا (۴۰۹۶ ویژگی) ضعیف عمل می‌کند. به این دلیل که در فضاها با ابعاد زیاد، محاسبه فاصله بین نقاط معنای خود را از دست می‌دهد و همه نقاط تقریباً به یک اندازه از هم فاصله دارند. این پدیده به **curse of dimensionality** معروف است. در مقابل، SVM با استفاده از کرنل RBF داده‌ها را به فضای بالاتر منتقل می‌کند و مرز تصمیم بهتری پیدا می‌کند.

نتیجه‌گیری

در این پروژه یک سیستم دسته‌بندی تصویر دودویی با استفاده از SVM پیاده‌سازی شد که تصاویر خودرو و موتورسیکلت را از یکدیگر تشخیص می‌دهد.

مهم‌ترین یافته‌ها:

کرنل RBF با $C=10$ و $\gamma=0.001$ بهترین عملکرد را با دقت ۸۳/۶٪ روی داده‌های تست به دست آورد. کرنل linear با وجود دقت ۱۰۰٪ روی train، تنها ۷۲٪ روی test داشت که نشان‌دهنده **overfitting** شدید است. مقایسه با KNN نشان داد که SVM با اختلاف ۱۲٪ عملکرد بهتری دارد.

چالش‌های پروژه:

زمان آموزش طولانی: هر مدل به طور میانگین ۳ دقیقه زمان برد
وجود **overfitting** در تمام مدل‌ها به دلیل استفاده از پیکسل خام
تنوع فرمت تصاویر jpg، jpeg، png که نیاز به مدیریت خطا داشت